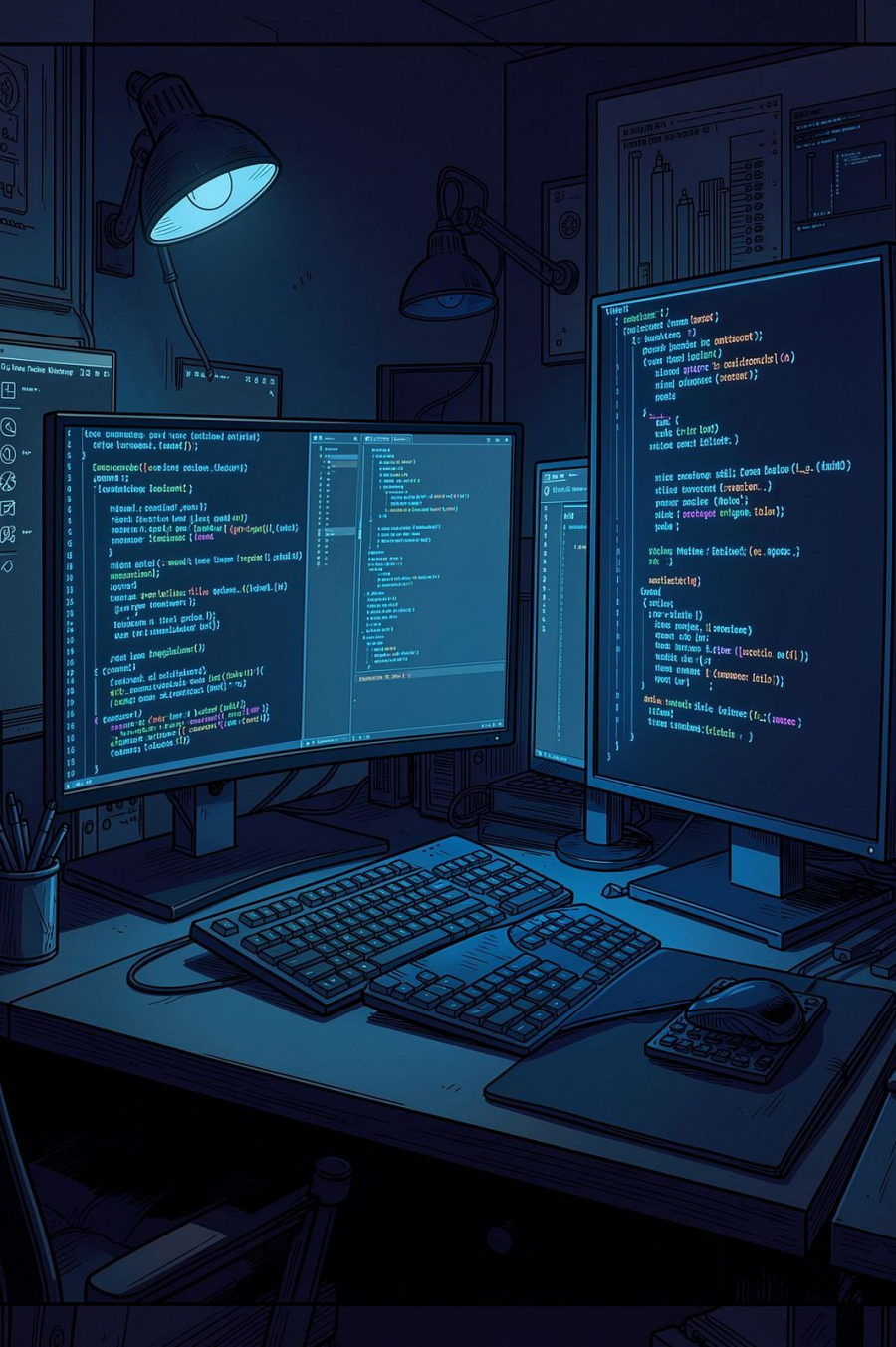




Les bonnes pratiques de programmation

Principes de génie logiciel transversaux — Java, Python, JavaScript

Un cours pour comprendre le *pourquoi* derrière chaque règle. Pas la syntaxe — le **raisonnement**.

Philosophie du cours

📄 **Idée centrale** : Une bonne pratique sans raison n'est qu'une contrainte. Chaque règle de ce cours s'appuie sur un raisonnement technique vérifiable.

Java / Python / JS

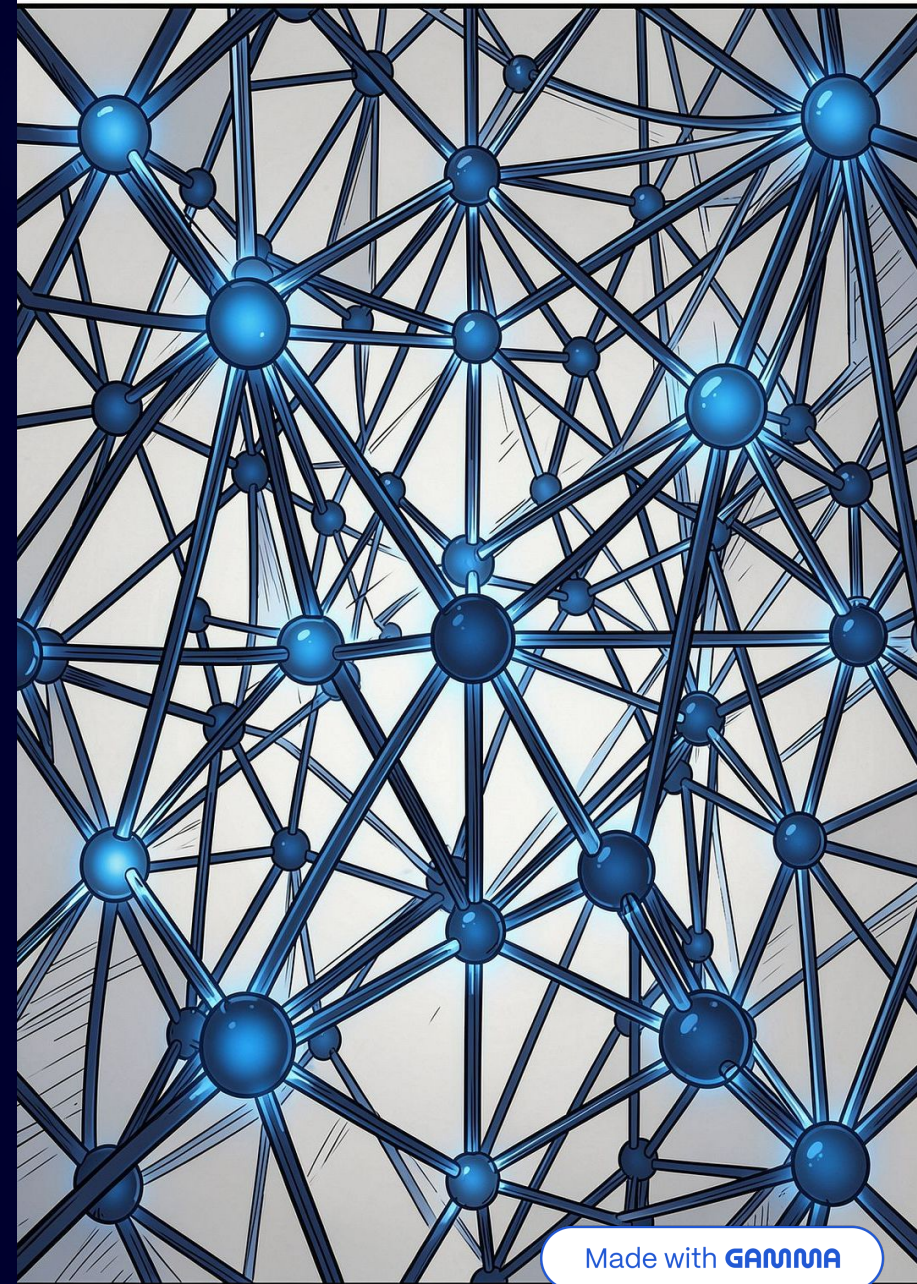
Trois langages, un même socle de principes. La rigueur se transfère.

Pourquoi avant comment

Comprendre le raisonnement, pas mémoriser des règles.

Transversal

Ce qui reste vrai quel que soit le langage.

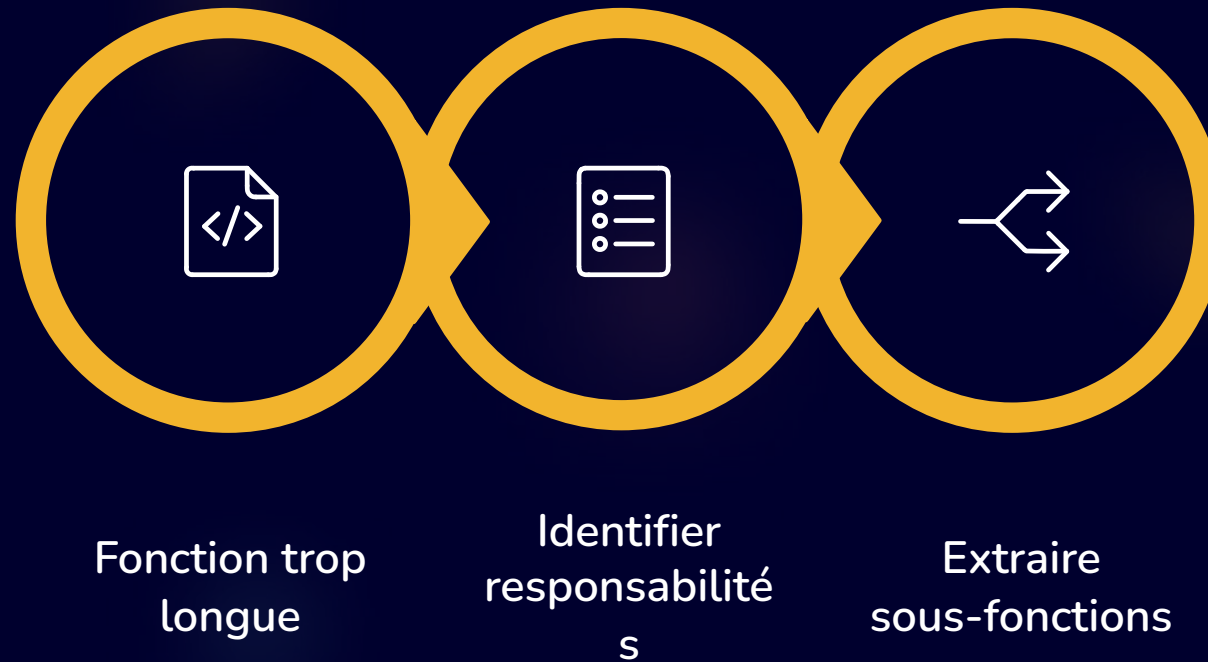


CHAPITRES 3-4

Lisibilité & Debugging

/*

Responsabilités et découpage



Une fonction qui fait cinq choses est cinq fois plus difficile à tester, déboguer et réutiliser. Le découpage n'est pas une contrainte stylistique – c'est une **nécessité technique**.

📌 **À retenir** : Si vous ne pouvez pas nommer une fonction clairement, c'est qu'elle fait probablement trop de choses.

CHAPITRE 6

Conditions et structures de contrôle

/*

Gestion des retours de fonction

Point de sortie unique

Un seul `return` en fin de fonction facilite le débogage et les tests unitaires. La variable de résultat porte un nom explicite.

📌 **Exception – Guard clauses** : Les retours anticipés sont acceptés pour éliminer les cas d'erreur en début de fonction. Ils *réduisent* la complexité, ne l'augmentent pas.

```
/* ✓ Point de sortie unique */  
BigDecimal computeDiscount(Order o) {  
    BigDecimal rate = BigDecimal.ZERO;  
    if (o.isPremium()) rate = TEN_PERCENT;  
    if (o.amount() > 1000) rate =  
        rate.add(FIVE_PERCENT);  
    return o.amount().multiply(rate);  
}
```

```
/* ✓ Guard clause acceptable */  
void process(User u) {  
    if (u == null) return; // guard  
    // logique principale...  
}
```

Éviter la duplication & gérer les valeurs



Centraliser la logique commune

Une règle dupliquée est une règle multipliée par le nombre de ses copies. Toute modification devient risquée.

Éliminer les valeurs magiques

0.15 → `VAT_RATE`. Un nom explique le *pourquoi* d'une valeur.

Choisir la bonne structure

Python `list` vs Java `ArrayList` vs `LinkedList`. Primitifs vs wrappers : `int` vs `Integer` a un coût mémoire réel.



Ne pas optimiser sans mesurer. L'optimisation prématurée est la racine de nombreux maux.

CHAPITRES 10-11

Commentaires, testabilité & maintenabilité

Le commentaire utile

Expliquer le **pourquoi**, pas le *quoi*. Le code dit déjà ce qu'il fait.

```
/*
```

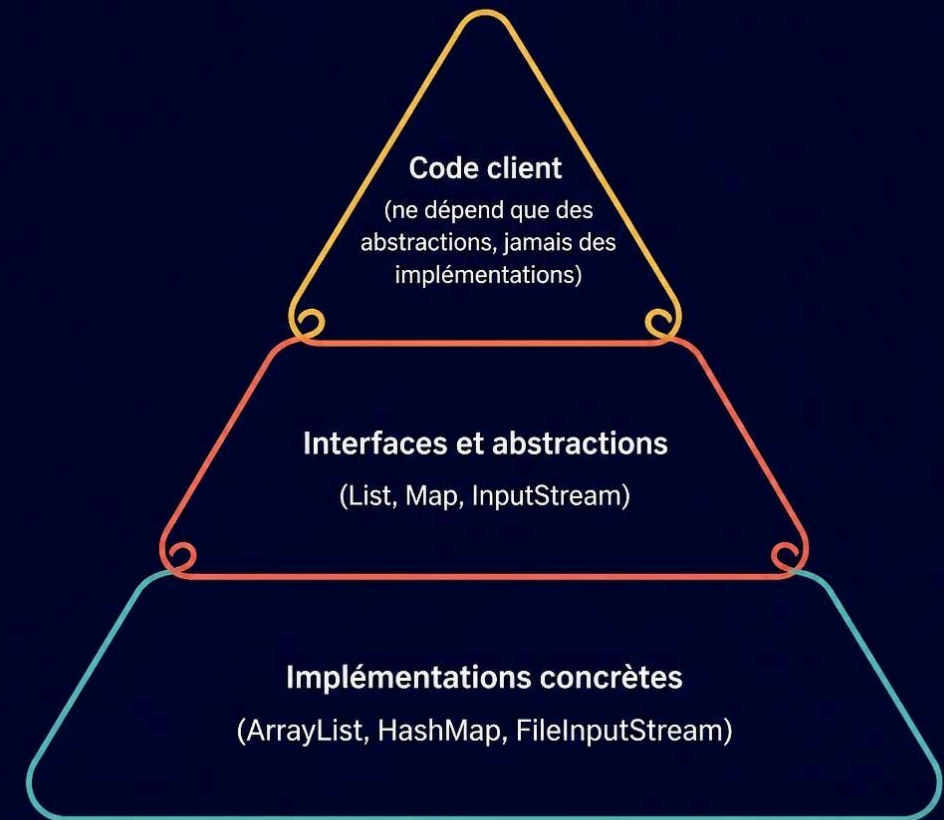

Abstraction, interfaces & architecture

Programmer contre une abstraction

List<T> plutôt qu'ArrayList<T>. Deque plutôt que ArrayDeque. On change l'implémentation sans casser le code client.

📄 **Architecture en points d'extension** : Comme une maison avec des prises électriques – on branche de nouvelles fonctionnalités sans casser les murs.

⚠️ **Attention aux casts** : Un (Type) fréquent signale souvent une abstraction insuffisante.



Synthèse — Checklist du développeur

Ces principes restent vrais **quel que soit le langage**. Une bonne pratique doit toujours avoir une raison.

01

Lisibilité

Une instruction par ligne, noms explicites, conventions respectées.

02

Découpage

Une fonction = une responsabilité.
Conditions décomposées.

03

Retours

Point de sortie unique — sauf guard clauses légitimes.

04

DRY & valeurs

Logique centralisée, constantes nommées, structures adaptées.

05

Abstraction

Programmer contre des interfaces, pas des implémentations.

📌 **Phrase directrice** : Écrire du code que quelqu'un d'autre peut lire, comprendre et modifier — y compris vous dans six mois.